

REACT JS

COMPLETE NOTES.

ATUL KUMAR (LinkedIn / Twitter)
NOTES GALLERY / CODING BUCS (Telegram)

→ What is ReactJS?

→ ReactJS is an open source Javascript library used for building interfaces. It focuses on creating dynamic interactive web applications by allowing developers to create reusable UI components.

→ Why we ReactJS?

→ Reacts is popular and widely used for several reasons:

Component Based Architecture

React allows you to break down your UI into reuse components. Making it easier to manage.

Efficient Updates:

It uses Virtual DOM, which optimized rendering by updating only parts of pages that change improving performance.

Large Ecosystem:

React has vast ecosystem of libraries and tools.

Cross Platform:

React can be used for both web and mobile app development.

PROS AND CONS OF REACTJS

PROS:

Component-based React encourages a modular, component based approach to build user interfaces.

Easy to learn and use: React is much easier to learn and use. It comes with good supply of documentation.

Reusable Components: A ReactJS web application is made up of multiple components. Each component has its own logic.

Scope for testing Codes : ReactJS applications are extremely easy to test. It offers scope where developers can test and debug their codes with help of native tool.

CONS:

Atul Kumar (LinkedIn/Twitter)
NOTES GALLERY / CODING BUGS (Telegram)

Poor Documentation React Technologies updating and accelerating so fast that there is no time to make proper documentation.

View Part ReactJS covers only UI layers of app & nothing else. so you still need to choose other technologies to get complete tooling set for development.

DIFFERENCE B/W REACTJS & ANGULAR

Field	ReactJS	Angular
used as	React js is a library (Javascript)	Angular is a framework.
Released	It was released in 2013.	It was released in 2010.
Written	Reactjs written in javascript.	Angular is written in microsoft's Typescript.
Routing	Routing is not easy in ReactJS.	Routing is easy compared to ReactJS.
Scalable	It is highly scalable.	It is less scalable.
Dom	It has a Virtual Dom.	It has regular Dom.
Testing	It supports unit testing	It support unit + integration testing.
Data Binding.	It supports uni directional	It supports bi directional.

DIFFERENCE BIW REACTJS and REACTNATIVE

REACTJS	REACTNATIVE
Installation Process React library is installed via npm package manager.	Installation Process. ReactNative is a command line interface tool requires both Node js and Reactnative cli to be installed.
Efficiency ReactJS is more efficient interms of code reuse ability.	Efficiency. ReactNative is more efficient interms of Performance & memory usage.
Technology Base ReactJS is javascript library used for building user interfaces.	Technology Base. ReactNative is a cross platform mobile development.
Components ReactJS Components are typically written in HTML.	Components ReactNative Components are written in JSX.

REACTJS	REACTNATIVE
<u>Storage</u> ReactJS is a good choice for projects that require high performance	<u>Storage</u> It is a good choice for Projects that need to be able to scale easy.
<u>Search engine friendly.</u> It is more search engine friendly than React Native. Platform.	<u>Search engine friendly</u> It cannot be made search engine friendly.
<u>Navigation.</u> ReactJS uses traditional browser based approach.	<u>Navigation.</u> ReactNative relies on native platform.
<u>Platform.</u> React is a framework for building application using javascript.	<u>Platform</u> It allows building native and cross platform mobile apps.
<u>Rendering</u> Browser code is rendered through virtual DOM.	<u>Rendering.</u> It uses native API to render all components.
<u>Syntax</u> Makes use of HTML and its syntax flow.	<u>Syntax.</u> ReactNative uses ReactNative syntax.

Key difference between React and Vue.

The main distinction between vue and React is how they approach application design.

While React focuses on creating reusable UI components vue takes approach by providing developers with frontend tools.

let now take a look at some particular differences.

VUE	REACT
It was single file components (SFC) to build different components.	It uses JSX as a Component format.
It is used to develop web-based application.	It is used to develop web as well as mobile application.
State Management library is called the Vuex	State Management library is called Redux.

VUE

The Performance is slow as react.

It is not suitable for long term support.

REACT

The Performance is slow when Compared to Vue.

It is suitable for long term support.

REACT JSX

ATUL KUMAR (Linkedin / Twitter)

NOTES GALLERY / CODING BUGS (Telegram)

What is JSX?

JSX stands for javascript XML.

JSX allows us to write HTML in React.

JSX makes it easier to write and add HTML in react.

Coding JSX

Jsx allows us to write HTML elements in Javascript and place them in DOM without Creating createElement() or appendChild()

Jsx Converts HTML tags into react elements. you are not required to use JSX, but JSX makes it easier to write React application.

Example 1

JSX:

```
const my element = <hi> I love JSX! </hi>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(my Element);
```

Output:-

I Love JSX!

Example 2

Without JSX:

```
const myElement = React.createElement('hi', {}, 'I do not');
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(myElement);
```

Output:-

I do not

In Example 1. JSX allows you to write HTML directly within Javascript.

JSX is an extension of Javascript language based on ES6.

Expression in JSX

With JSX you can write expression inside curly braces `{ }`. The expression can be react variable or property. JSX will execute expression and return the result.

Example

Execute the expression `5+5`
`const myElement = <h1> React is {5+5} times better with JS </h1>;`

Output

React is 10 times better with JSX

Inserting a large block of HTML

To write HTML on multiple lines, put HTML inside Paranthesis.

Example:

```
const myElem = (
  <ul>
    <li> Apple </li>
    <li> Banana </li>
    <li> cherries </li>
  </ul>
);
```

Output:-

- Apple
- Banana
- cherries

Example:-

```
const myElement = (
  {div} = (
    {P} I am Paragraph {/P}
    {P} Paragraph too {/P}
  {/div}
);
```

⚡ JSX will throw error if HTML is not correct or if HTML misses a parent element. Here {div} is Parent {P} is child.

Output:-

I am Paragraph

Paragraph too

Elements must be closed in
close empty elements with / }

Setting up a React Environment

If you have npx and node.js installed, you can create a React application by using create-react-app.

The create-react-app will set up everything you need to react application to run.

Run the React Application
cd my-react-app

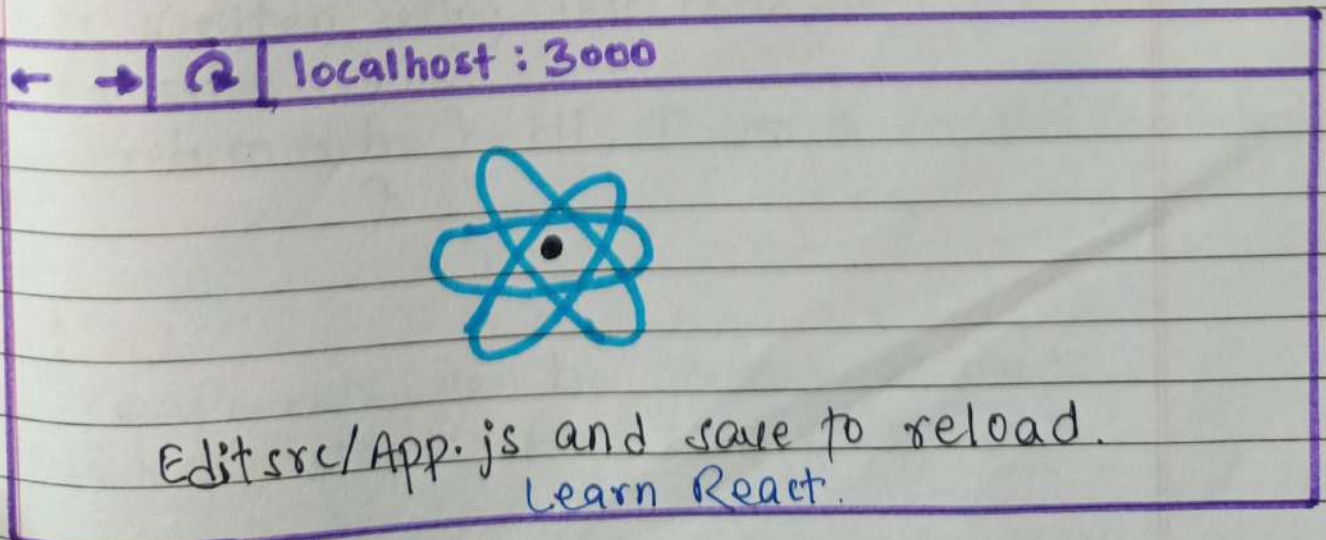
Run this command to run react application
npm start.

A new browser will pop up with your newly created ReactApp!

If not open browser and type localhost
:3000

ATUL KUMAR (Linkedin/Twitter)
NOTES GALLERY / CODING BULBS (Telegram)

The result:



REACT Components

When creating a React Component, the component name must start with uppercase letter.

Class Component must include extends. React. Component statement.

The component also requires render(). This method returns HTML.

Example:-

```
class Car extends React.Component {
  render() {
    return <h2> Hi, I am a car! </h2>
  }
}
```

Function Component

Same as react component, only difference is written using less code.

```
function () {
  return <h2> Hi, I am a car! </h2>;
}
```

Props

Components can be passed as props, which stands for Properties.

Component in files

React is all about re-using code and it is recommended to split your components into separate files.

To do that Create a new file with .js files.

React class Component State

React class components have built in state object.

Creating the State Object

```

class Car extends React.Component {
  constructor (props) {
    super (props);
    this.state = { brand: "ford" };
  }
  render () {
    return (
      <div>
        <h1> My Car </h1>
      </div>
    );
  }
}

```

Using the state Object.

Refer to the state Object anywhere in the Component using this.state.propertyname syntax.

Example:-

```

class Car extends React.Component {
  constructor (props) {
    super (props);
    this.state = {
      brand: "Ford"
      Model: "Mustang"
      color: "red"
      year: 1964
    };
  }
}

```

ARUL KOMAR (Linkedin/Twitter)
NOTES GALLERY/COPING BUGS (Telegram)

```

render () {
  return (
    <div>
      <h>My {this.state.brand} </h>
      <p>
        It is a {this.state.color}
        {this.state.model}
        From {this.state.year}
      </p>
    </div>
  );
}
}

```

Outputs:-

MY FORD
It is a red Mustang from 1964

Changing the state object

To change a value in state object, use the `this.setState()`

Example:-

Class Car extends React.Component {
 constructor (props)

super (props);

this.state = {

brand = "Ford",

Model = "Mustang",

color = "red",

year = 1960

};

}

changeColor = () => {

this.setState ({ color: "blue" });

}

render () {

return (

<div>

```
<h1> My <-this.state.> </h1>
<p>
```

```
  It is a {this.state.color}
  {this.state.model}
  from {this.state.year}
</p>
```

```
< button >
```

```
  type = "button"
```

```
  onClick = {this.changeColor}
```

```
> change color </button>
```

```
</div>
```

```
);
```

```
}
```

```
}
```

Output:-

My Ford

It is a red Mustang from 1964

change color

(When you click)

My Ford

It is a blue Mustang from 1960

Change color

LIFECYCLE OF COMPONENTS

Each Component in React has a lifecycle which you can monitor and manipulate during three main phases.

The three phases are:

- Mounting
- Updating
- Unmounting

Mounting

Mounting means putting elements in DOM.

React has four built in methods that gets called, when mounting a component:

1. Constructor ()
2. getDerivedStateFromProps ()
3. render ()
4. componentDidMount ()

ATUL KUMAR (LinkedIn/Twitter)
NOTES GALLERY / CODING BULBS (Telegram)

Constructor

The Constructor Method is called by React, every time you make a Component.

Example:-

```
class Header extends React.Component {
  constructor (props) {
    super (props);
    this.state = { favourite color : "red" };
  }
  render () {
    return {
```

```
    <h1> My favourite color is { this.state.
      favouritecolor } </h1> );
  }
}
```

```
const root = ReactDOM.createRoot (document.
  getElementById ('root'));
root.render (<Header />);
```

Output:-

My Favourite Color is red

getDerivedStateFromProps

The `getDerivedStateFromProps()` is called right before element(s) in the DOM.

The example below starts with favourite color being "red", but the `getDerivedStateFromProps()` updates the favourite color based on `favcol` attribute.

Example:-

```

class Header extends React.Component {
  constructor ( props ) {
    super ( props ) ;
    this.state = { favouritecolor : "red" } ;
  }
  static getDerivedStateFromProps ( Props, state ) {
    return { favouritecolor : props.favcol } ;
  }
  render () {
    return (
      < h1 > My Favourite Color is { this.state.
        favourite color < / h1 >
    ) ;
  }
}

```

```
const root = ReactDOM.createRoot (document.  
    getElementById ("root"));  
root.render ( < Header.favcol = "yellow" / > );
```

Output:-

My Favourite Color is yellow

Render

ARUL KUMAR (Linkedin/Twitter)
NOTES GALLERY/CODING BUCS (Telegram)

render() is required and is the method that actually outputs HTML to the Dom.

Example:-

```
class Header extends React.Component {  
  render () {  
    return (  
      < h1 > This is the Component of Header </h1>  
    );  
  }  
}
```

```
const root = ReactDOM.createRoot (document.  
    getElementById ("root"));
```

```
root.render ( < Header / > );
```

Output:-

This is the Component of Header

Component DidMount

ComponentDidMount() method is called after the component is rendered.

Example:-

At first my favourite color is red, but give me a second, and it is yellow instead.

class Header extends React. Component {

constructor (props) {

super (props);

this.state = { favourite color : "red" };

}

componentDidMount () {

setTimeout () = > {

this.setState ({ favourite color : "yellow" })

} , 1000)

}

```
render() {  
  return (  
    <h1> My favourite color is { this.state.  
      favourite color } </h1>  
  );  
}
```

```
const root = ReactDOM.createRoot( document.  
  getElementById( 'root' ) );
```

Output :-

My favourite color is yellow

Updating

The next phase in lifecycle is when component is updated.

React has five built in methods that gets called when component is updated.

React Props

Props are arguments passed into React components.
props stands for properties.

Example:-

Add a brand name attribute to car element
const MyElement = <Car brand = "Ford" />;

Example:-

Use brand attribute in the example

```
function Car (props)
return <h2> I am a { props.brand } ! </h2>;
const MyElement = <Car brand = "Ford" />;
```

```
const root = ReactDOM.createRoot (document.
getElementsByById ('root'));
root.render (myElement);
```

Output:-

I am a Ford

React Events

React has same events as HTML: click, change, mouseover etc.

ATUL KUMAR (LinkedIn / Twitter)
NOTES GALLERY / CODING BUGS (Telegram)

Adding Events

React events are written in camelcase syntax on click instead onclick

React event handler are written inside curly braces

onclick = { shoot } instead of
onclick = "shoot ()"

React

```
< button onclick = { shoot } > Take shot!  
  < / button >
```

HTML

```
< button on click = "shoot ()" > Takeshot!  
  < / button >
```

Passing Arguments

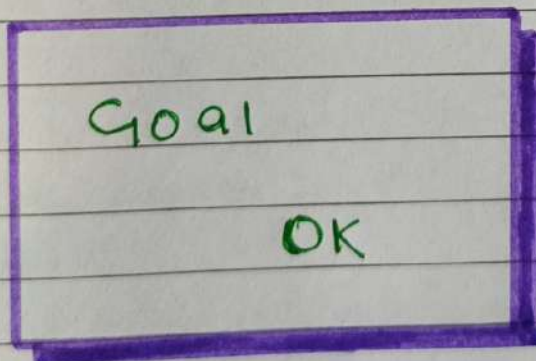
To pass an argument to event handler use an arrow function.

Example:-

Send "Goal" as parameter to shoot function

```
function Football () {  
  const shoot = (a) => {  
    alert (a);  
  }  
  return (  
    < button on click = { () => shoot ("Goal")  
    } > Take shot!  
    < / button >  
  );  
}
```

```
const root = ReactDOM.createRoot (  
  root.render ( < Football / > );
```

Output:-

React Conditional Rendering

IF Statement

Example

```
function MissedGoal () {  
  return <h1> MISSED! </h1>;  
}
```

Logical & Operator

We can embed javascript expression in JSX by using curly braces.

```
function Garage (props) {  
  const cars = props.cars;  
  return (  
    <>  
    <h1> Garage </h1>  
    { cars.length > 0 &&  
    <h2>  
      you have { cars.length } cars in your garage.  
    </h2>  
    }  
    </>  
  );  
}
```

```
const Cars = ['Ford', 'BMW', 'Audi'];
const root = ReactDOM.createRoot(document.
  getElementById('root'));
root.render(< Garage cars = { cars } />);
```

Output:-

Garage

You have 3 Cars in your garage

Ternary Operator

ARUL KOMAR (Linkedin/Twitter)
NOTES GALLERY/CODING BUSS (Telegram)

Condition ? true : false

Example:-

```
function Goal(props) {
  const isGoal = props.isGoal;
  return (
    <>
    { isGoal ? < MadeGoal /> : < MissedGoal /> }
    </>
  );
}
```

```
const root = ReactDOM.createRoot( document.  
  .getElementById('root'));  
root.render(<Goal is Goal = {false}/>);
```

Output :-

MISSED

React List

List are used to display data in Ordered format & mainly used to display menus on websites.

Example.

```
Var numbers = [1, 2, 3, 4, 5];  
const multiplyNum = numbers.map((number) =>  
{  
  return (number * 5)  
});  
console.log(multiplyNum);
```

Output :-

[5, 10, 15, 20, 25]

React Keys

A key is a unique identifier. In react, it is used to identify which items have changed, updated or deleted from the list.

Example:

```
const stringLists = ['Peter', 'sachin', 'Kelvin',  
  'Dhoni']  
const updatedLists = stringList.map((strList) => {  
  <li key={strList.id}> {strList} </li>,  
});
```

React Refs

Refs is shorthand used for reference. It is an attribute which makes it possible to store a reference to particular DOM nodes or React elements.

When to use Refs

When we need DOM measurements such as managing focus, text selection or media playback.

It can also use as in call backs.

When not to use Refs

Instead of using `open()` and `close()` methods on a Dialog Components, you need to pass an `is Open` prop to it.

How to access Refs

```
const node = this.callRef.current
```

Callback refs

Another way to using refs is callback ref and it gives more control when refs are set and unset.

React Forms

Arul Kumar (LinkedIn/Twitter)
NOTES GALLERY / CODING BUGS (Telegram)

Example:-

Add a form that allows user to enter name:

```
Function MyForm () {  
  return (  
    <form>  
      <label> enter name :  
      <input type = "text" />  
    </label>  
  </form>  
  )  
}
```

Output:-

Enter your name:

React Router

Add react router in your application, run this in terminal from root directory of application.

```
npm i -D react-router-dom
```

Basic usage

Now use router index.js file.

Example:-

```
export default function App() {
  return (
    <BrowserRouter>
      <Route path="/" element={ <Layout /> } />
      <Route index element={ <Home /> } />
      <Route path="blogs" element={ <Blogs /> } />
      <Route path="contact" element={ <Contact /> } />
      <Route path="*" element={ <NoPage /> } />
    </Route>
  )
}
```

```
</Router>  
</BrowserRouter>  
};  
}
```

```
const root = ReactDOM.createRoot (document.  
    getElementById ( 'root' ) );  
root.render ( < App / > );
```

Output:-

(localhost:3000)

Home

Blogs

Contact

Blog Articles

Benefits of React Router

- It is not necessary to set browser history manually link uses a navigate internal links in application.

- It uses switch features for rendering.
- If the router need only single child element.

React Memo

React Memo is Higher Order Component which itself wraps around component to rendered output

It is higher Order Component
uses last rendered result
only check for prop changes
Effect on React Hooks

Where to use React Memo

If you are using a Pure Functional Component:

Use it when you know before hand that a component will render quite often.

Use it if re-rendering is done using same Props.

Use it if your Component is big enough to have props equality check done by React with decent number of UI elements.

Where NOT to use React Memo?

You should not use React Memo in all React Component that implements the `shouldComponentUpdate()` method.

This is because it return true by default.

The render change that occurs by using React Memo is same implementation of `shouldComponentUpdate()` which does shallow comparison.

Other than this

- Don't use React Memo if Component isn't heavy.
- Wrapping a class based Component in React Memo is undesirable.

React Fragments

ATUL KUMAR (Linkedin / Twitter)
NOTES GALLERY / CODING BUGS (Telegram)

When you want to render something, you need to use `render()` inside Component.

This render method can return single or multiple elements.

If you want to return multiple elements. The render method requires 'div' tag and put entire content inside it.

This extra node to DOM sometimes results in wrong formatting of your HTML output.

Example:-

```
// Rendering with div tag
class App extends React.Component {
  render () {
    // div element
    <div>
      <h2> Hello world </h2>
    </div>
  };
}
```

To solve the above problem React introduced fragments.

Fragments allow you to group list of children without adding extra nodes to DOM.

Syntax:-

```
<React.Fragment>
  <h2> child1 </h2>
  <p> child2 </p>
</React.Fragment>
```

Example:-

```
class App extends React.Component {
  render() {
    return (
      <React.Fragment>
        <h2> Hello World </h2>
        <p> Welcome to TopperWorld </p>
      </React.Fragment>
    );
  }
}
```

Why we use Fragments

The main reason to use fragments is
It makes execution of code faster as
compared to div tag.
It takes less memory.

Fragment short Syntax

There is also another shorthand exists for
declaring fragments. It looks like empty
tag in which we can use '< >' and '' instead
of 'React.Fragment'.

Example:-

```
class Columns extends React.Component {
  render () {
    return (
      < >
      <h2> Hello </h2>
      <p> Welcome to Topperworld </p>
      </>
    );
  }
}
```

ATUL KOMAR (LinkedIn/Twitter)
NOTES GALLERY/CODING BUGS (Telegram)

Keyed Fragments

The shorthand syntax does not accept key attributes. Key is the Key Only attribute that can be passed with the fragments.

You need a key for mapping a collection to an array of fragments such as to create a description list.

If you need to provide keys you have to declare fragments with explicit syntax.

```

Function = (Props) {
  return (
    < Fragment >
    { props.items.data.map ( item => (
      // without 'Key' React will give warning
      < React.Fragment Key = { item.id } >
      < h2 > { item.name } < / h2 >
      < p > { item.url } < / p >
      < p > { item.description } < / p >
    < / React.Fragment >
    ) ) }
    < / Fragment >
  )
}

```

Difference between state and Props

<u>Props</u>	<u>state</u>
The data is passed from one component to another.	The data is passed within the component.
Props can be used within state and functional components.	The state can be used only with state component / class component.

Props are read only.

The state is both read and write.

It is immutable.

It is mutable.

Props are controlled by whoever renders the Component.

State is controlled by react Component.

Props can be accessed in functional Component using props parameter and in - class Component props can accessed using this Props.

State can be accessed using the state hooks in functional Components and in class Components can be accessed.

Props are read only.

State changes can be asynchronous.

Props Communicate between Components.

State display changes with component.

Controlled Component VS Uncontrolled Component

Uncontrolled

If accepts Current value as Props

It has better Control over form elements and data.

It does not maintain internal state.

Data is Controlled by Parent Component.

It allows validation control.

Styling using CSS (React)

There are many ways to style React with CSS.

Common ways are :-

- Inline Styling
- CSS stylesheets
- CSS Modules.

ATUL KUMAR (LinkedIn/Twitter)
 NOTES GALLERY / CODING BUGS (Telegram)

Inline Styling :-

To style an element with inline styling attribute.

Value must be javascript Object.

Example :-

```
Const Header = ( ) => {
  return (
    <>
    <h1 style = < { color: "red" } > Hello Topper
    </h1>
    <p> welcome to Topperworld! <p>
    </>
  );
}
```

```
const root = ReactDOM.createRoot(document.  
  getElementById('root'));  
root.render(<Header/>);
```

Output:-

Hello TOPPER
welcome to TOPPERworld !

Note:-

In JSX, javascript expression are written inside curly braces, and since javascript objects also uses curly braces, styling above written inside two set of curly braces.

Camel Case Property Names.

Example:-

Use backgroundColor instead of background-color :-

```
const Header = ( ) => {  
  return (  
    <>
```

```
<h1 style = { { backgroundColor : "lightblue"
                } } >
    Hello </h1>
<p> Add style </p>
</>
);
}
```

```
const root = ReactDOM.createRoot ( document.
    getElementById ('root')));
root.render (< Header /> );
```

Output:-

Hello
Add style

Javascript Object

You can also create object with styling.

Example:-

Create style object named mystyle:-

```
const Header = () => {
    const mystyle = {
```

```

color : " white "
font-family : " Sans - Serif "
background color : " red "
};
return (
  <>

```

```

<h1> style = { mystyle } > Hello </h1>
<p> Welcome to React </p>
</>
);
}

```

```

const root = ReactDOM.createRoot( document
    .getElementById( 'root' ) );
root.render( < Header / > );

```

Output :-

Hello
Welcome to React

CSS Stylesheet

Write your CSS styling in .css file extension and import it in your application.

Create App.css and insert some CSS code inside it.

```
App-Css
body {
background-color : #28ac34;
color : white;
Font family : sans-serif;
}
```

CSS Modules

Atul Kumar (Linkedin / Twitter)
NOTES GALLERY / CODING BUCS (Telegram)

Another way of adding styles to your application is using CSS modules.

CSS modules are convenient for components that are placed in separate files.

Create CSS module with .module.css extension example:- my-style.module.css.

My-style.module.css

```
.bigblue {
color : DodgerBlue;
font-family : sans-serif;
text-align : center;
padding : 40px;
}
```

Styling React Using Sass

- What is Sass?
- Sass is a CSS - Processor
Sass files are executed on server and sends CSS to browser.

Install Sass by running this Command on your terminal

```
> npm i sass
```

Create a sass file with .sass.

Example:-

My - sass.scss

Create a variable to define color of text

```
$ myColor: red;
h1 {
  color: $myColor;
}
```

React Hooks

What are react hooks ?

React Hooks are a new feature of react-js using this it is possible to use state and other react features without writing class.

Rules to use hooks

Hooks should be used in topmost scope of code and never be used within loops, conditions or nested functions.

Hooks should only be used by react function components.

Don't use ordinary javascript methods to call hooks.

These rules are also applicable for custom hooks.

React has also built in hooks. ie useEffect and useState.

Use State Hook

The use state hook is used for storing a state within a component.

The useState hook allows you to store and access state inside a component without using `this.state` or `this.setState()`.

UseEffect Hook

ATUL KUMAR (LinkedIn/Twitter)
NOTES GALLERY / CODING BUGS (Telegram)

It gives function components the ability to perform side effect, resulting in accomplishing the same thing that `ComponentDidMount`, `ComponentDidUpdate` and `ComponentWillUnmount` do in React classes, but with single API.

Custom Hooks.

Custom hooks is an effective option in case where we want to implement derived functionality of both `useState` and `useEffect`.

Benefits of React Hooks

Easy to Understand Complex Components -
 Reduced Complexity without classes.
 Easy to reuse Stateful Logic.

React Flux vs MVC

MVC stands for Model View Controller - It is an architectural pattern used for developing the user interface.

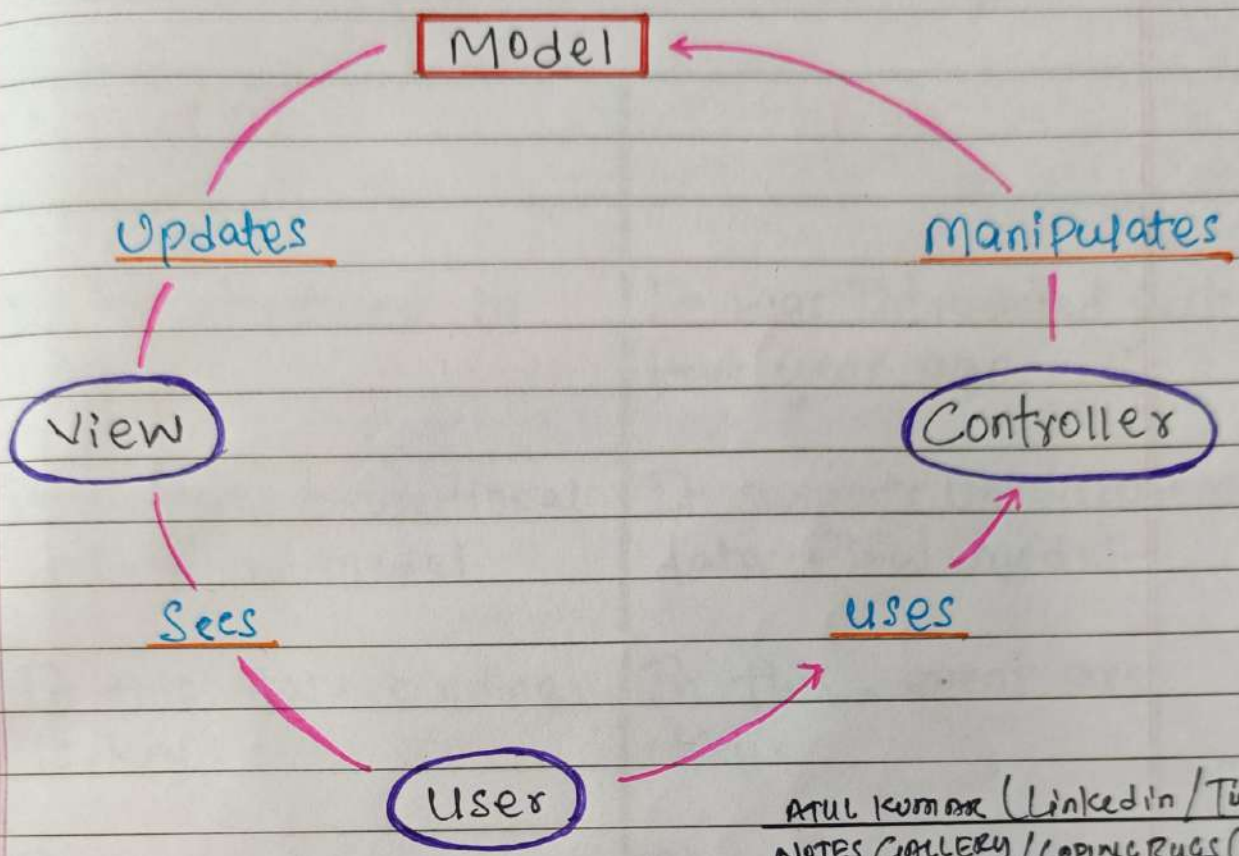
MVC Architecture

Model :- It is responsible for maintaining the behaviour and data of an application.

View :- It is used to display Model in user interface.

Controller :- It acts an interface between Model and view components.

It takes user input, manipulates the data and causes view.



ATUL KUMAR (LinkedIn/Twitter)
 NOTES GALLERY / COPIING BUGS (Telegram)

MVC architecture.

Flux:

Flux is an application architecture that Facebook uses for building client-side web application.

Flux architecture has 3 major rules in dealing with data.

- Dispatcher
- Store
- Views

Difference between MVC and Flux

MVC	FLUX
It is introduced in 1976.	It was Introduced just few year ago.
It supports bidirectional data flow model.	It supports Unidirectional data flow model.
In this, data binding is key.	In this, event are Key.
It is synchronous	It is asynchronous
It is hard to debug.	It is easy to debug.
It is difficult to Understand as Project size increases.	It is easy to Understand.
Testing of application is difficult.	Testing of application is easy.
Scalability is Complex	Scalability is easy.

React Redux

React Redux is a predictable state container for javascript application.

It helps you write apps that behave consistently run in different environments (client, server, native) and are easy to test.

Redux is a state management tool

Redux can be used with any javascript framework.

Redux stores the state of application, and the components can access the state from state store.

Principle of Redux.

The three most important redux principles are:-

- Single source of Truth.
- State is Read Only.
- changes are made with Pure functions.

Single Source of Truth.

The state of your whole application is stored in an object tree within Single-store.

A Single state tree makes it easier to debug or inspect an application.

It gives you faster development of cycle by enabling you to persist in you app's navigation state.

State is read only.

ATUL KUMAR (Linkedin/Twitter)
NOTES GALLERY / CODING BULLS (Telegram)

The Only way to change the state to initiate an action an Object describing.

This feature ensures that no events like network callback or views can change state.

Actions are just plain Objects, they can be logged.

Changes are made with Pure functions.

To specify how actions transform state tree. The user can return new state objects instead of mutating previous state.